

Phase 6 Structured Notes: Circular Queue

1. Current Progress Before Phase 6

Before Phase 6, you already learned the main queue ideas.

In **Phase 1**, you learned that a queue follows this rule:

FIFO = First In, First Out

This means:

The first value inserted into the queue is the first value removed from the queue.

You also learned that a normal queue has two important ends:

Queue Part	Simple Meaning
front	The side where deletion happens
rear	The side where insertion happens

So in a normal queue:

Insert at rear.
Delete from front.

In **Phase 2**, you compared two array queue designs:

Design	Main Idea	Result
Single-pointer queue	Uses only rear	Deletion is slow because elements must shift
Two-pointer queue	Uses front and rear	Deletion is fast because only front moves

The most important Phase 2 idea was:

The two-pointer queue avoids shifting.

In **Phase 3**, you built a complete linear array queue in C using:

- struct Queue
- size
- front
- rear
- Q
- create()
- enqueue()
- dequeue()
- display()
- malloc()
- free()

You also learned these important conditions:

```
front == rear    // queue is empty
rear == size - 1 // queue is full in a linear array queue
```

In **Phase 4**, you wrote the same queue idea using C++ OOP.

You used:

- class Queue
- private data
- public methods
- constructors
- destructor
- new[]
- delete[]
- object syntax like `q.enqueue(10)`

In **Phase 5**, you learned the main drawback of a two-pointer linear queue.

The problem was not speed.

The problem was **memory reuse**.

A linear queue is fast, but it wastes memory because:

```
rear only moves forward.
```

That means deleted spaces at the beginning of the array cannot be reused.

This caused the **full-but-empty problem**.

Now Phase 6 gives the real solution:

Circular queue.

2. Main Goal of Phase 6

The goal of Phase 6 is to learn how a circular queue solves the memory-wastage problem of a linear queue.

A circular queue allows `rear` and `front` to move around the array in a circle.

Instead of stopping at the last index, the queue can return to the beginning.

The main idea is:

Treat the array like a circle, not like a straight line.

3. What Is a Circular Queue?

A **circular queue** is a queue where the last position of the array is connected back to the first position.

A normal linear queue moves like this:

0 → 1 → 2 → 3 → 4

When `rear` reaches index `4`, it stops.

A circular queue moves like this:

0 → 1 → 2 → 3 → 4 → 0 → 1 → 2 ...

So after the last index, the next position becomes index `0` again.

This is called **wrap-around behavior**.

4. Why Do We Need a Circular Queue?

We need a circular queue because a linear queue wastes space.

Example with a linear queue of size 5:

```
Index:  0   1   2   3   4
Array: [10][20][30][40][50]
```

Now delete 10 and 20.

The logical queue becomes:

```
30 40 50
```

But the array still looks like this:

```
Index:  0   1   2   3   4
Array: [10][20][30][40][50]
```

Indexes 0 and 1 are no longer part of the queue.

They are logically empty.

But in a linear queue, rear is already at the last index.

So the queue cannot insert new values, even though there are empty spaces at the beginning.

A circular queue fixes this by allowing rear to go back to index 0.

5. Beginner Analogy

Imagine a circular table with 5 chairs.

```
Chair 0, Chair 1, Chair 2, Chair 3, Chair 4
```

If someone leaves Chair 0, another person can later sit there again.

The table does not end at Chair 4.

After Chair 4, we can return to Chair 0.

That is how a circular queue works.

6. The Most Important Formula in Circular Queue

The most important formula in Phase 6 is:

```
next = (current + 1) % size;
```

This formula gives the next circular position.

The `%` symbol is called the **modulo operator**.

It gives the remainder after division.

7. Why Modulo Is Used

Modulo helps us return to the beginning of the array after reaching the end.

Example with `size = 5`:

Current Position	Formula	Next Position
0	<code>(0 + 1) % 5</code>	1
1	<code>(1 + 1) % 5</code>	2
2	<code>(2 + 1) % 5</code>	3
3	<code>(3 + 1) % 5</code>	4
4	<code>(4 + 1) % 5</code>	0

The last row is the key idea:

```
After index 4, the next position becomes index 0.
```

That is how wrapping happens.

8. Circular Movement Example With Size 7

If the size is 7, valid indexes are:

0 1 2 3 4 5 6

Circular movement works like this:

Current Position	Formula	Next Position
0	$(0 + 1) \% 7$	1
2	$(2 + 1) \% 7$	3
5	$(5 + 1) \% 7$	6
6	$(6 + 1) \% 7$	0

So when the current position is 6, the next position becomes 0.

9. Main Difference Between Linear Queue and Circular Queue

Idea	Linear Queue	Circular Queue
Movement	Straight line	Circle
After last index	Stops	Goes back to index 0
Memory reuse	Poor	Better
Uses modulo	No	Yes
Main problem solved	None	Solves wasted front spaces

A linear queue moves like this:

0 → 1 → 2 → 3 → 4

A circular queue moves like this:

0 → 1 → 2 → 3 → 4 → 0

10. Circular Queue Structure

The circular queue uses the same basic structure as the array queue:

```
struct Queue {  
    int size;  
    int front;  
    int rear;  
    int *Q;  
};
```

11. Meaning of Each Structure Member

Member	Simple Meaning
size	Total array size
front	Position used for deletion
rear	Position used for insertion
Q	Pointer to the array that stores queue values

Important point:

front and rear are index values.
They are not actual memory addresses.

Example:

```
front = 2  
rear = 4
```

This means `front` and `rear` are positions inside the array.

12. Starting State of a Circular Queue

In a linear queue, we usually started with:

```
front = rear = -1;
```

But in this circular queue design, we start with:

```
front = rear = 0;
```

This means:

```
front = 0  
rear = 0
```

The queue is empty because:

```
front == rear
```

13. Why Circular Queue Starts at 0

In this design, `front` and `rear` both start at `0` because we use the empty-slot rule.

At the beginning:

```
front = 0  
rear = 0
```

Since both are equal, there are no valid elements in the queue.

So:

```
front == rear means empty.
```

14. The Empty-Slot Rule

This circular queue design keeps one array space empty.

This is called the **empty-slot rule**.

It means:

One position is always left unused.

So if the array size is `5`, the queue can store only `4` values.

If the array size is `7`, the queue can store only `6` values.

In general:

Usable capacity = size - 1

15. Why Do We Leave One Slot Empty?

We leave one slot empty so that we can clearly tell the difference between:

Queue is empty
Queue is full

Without this rule, both empty and full situations can become confusing.

Using the empty-slot rule:

`front == rear`

always means empty.

And:

`(rear + 1) % size == front`

always means full.

16. Empty Condition

The circular queue is empty when:

```
front == rear
```

Simple meaning:

```
There are no valid elements between front and rear.
```

Example:

```
front = 0  
rear = 0
```

The queue is empty.

Another example:

```
front = 3  
rear = 3
```

The queue is also empty.

17. Full Condition

The circular queue is full when:

```
(rear + 1) % size == front
```

Simple meaning:

```
If rear moves one more step, it will collide with front.
```

So we stop before that happens.

Example with `size = 5`:

```
front = 0  
rear = 4
```

Check full condition:

```
(rear + 1) % size == front
```

Substitute values:

```
(4 + 1) % 5 == 0
```

```
5 % 5 == 0
```

```
0 == 0
```

So the queue is full.

18. Important Conditions Summary

Condition	Meaning
<code>front == rear</code>	Queue is empty
<code>(rear + 1) % size == front</code>	Queue is full
<code>(rear + 1) % size</code>	Next rear position
<code>(front + 1) % size</code>	Next front position

19. Important Valid Range Idea

In this circular queue design, `front` does not directly point to the first valid element.

Instead:

```
front points before the first valid element.
```

So the first valid element is found at:

```
(front + 1) % size
```

The last valid element is at:

```
rear
```

This is similar to the earlier linear queue idea, but now we use modulo.

20. The create() Function

The `create()` function prepares the circular queue.

```
void create(struct Queue *q, int size) {
    q->size = size;
    q->front = q->rear = 0;
    q->Q = (int *)malloc(q->size * sizeof(int));
}
```

21. Job of create()

The `create()` function does three main things:

1. Stores the queue size.
2. Sets `front` and `rear` to `0`.
3. Creates the array in heap memory using `malloc()`.

After calling:

```
create(&q, 5);
```

The queue state is:

```
size = 5
front = 0
rear = 0
Q = memory for 5 integers
```

The queue is empty.

22. Line-by-Line Explanation of create()

Line 1

```
q->size = size;
```

This stores the array size inside the queue.

Line 2

```
q->front = q->rear = 0;
```

This makes the queue empty.

Why?

Because:

```
front == rear
```

Line 3

```
q->Q = (int *)malloc(q->size * sizeof(int));
```

This creates memory for the queue array.

If `size = 5`, it creates space for 5 integers.

23. The enqueue() Function

`enqueue()` means:

```
Insert a value into the queue.
```

In a normal queue, insertion happens at the rear.

Circular queue insertion also happens at the rear, but `rear` moves circularly.

24. Circular Queue enqueue() Code

```
void enqueue(struct Queue *q, int x) {
    if ((q->rear + 1) % q->size == q->front) {
        printf("Queue is Full\n");
    } else {
        q->rear = (q->rear + 1) % q->size;
        q->Q[q->rear] = x;
    }
}
```

25. Logic of enqueue()

To insert a value:

1. Check whether the queue is full.
2. If the queue is full, print `Queue is Full`.
3. If the queue is not full, move `rear` to the next circular position.
4. Store the new value at `Q[rear]`.

26. Full Check in enqueue()

The full check is:

```
if ((q->rear + 1) % q->size == q->front)
```

This means:

If the next rear position is front, the queue is full.

We do not allow `rear` to move into `front`.

That is why one space remains empty.

27. Moving rear in enqueue()

This line moves `rear` :

```
q->rear = (q->rear + 1) % q->size;
```

Simple meaning:

Move rear one step forward.
If rear reaches the end, wrap it back to 0.

28. Storing the Value in enqueue()

After moving `rear`, we store the value:

```
q->Q[q->rear] = x;
```

Example:

```
rear = 2  
x = 20
```

Then:

```
Q[2] = 20;
```

29. enqueue() Dry Run

Suppose:

```
size = 5  
front = 0  
rear = 0
```

The queue is empty.

Now call:

```
enqueue(&q, 10);
```

Check full:

```
(rear + 1) % size == front
```

```
(0 + 1) % 5 == 0
```

```
1 == 0
```

False, so the queue is not full.

Move `rear` :

```
rear = (0 + 1) % 5 = 1
```

Store value:

```
Q[1] = 10
```

Now:

```
front = 0  
rear = 1  
Logical queue = 10
```

30. Another enqueue() Dry Run

Now call:

```
enqueue(&q, 20);
```

Before insertion:


```
front = 0  
rear = 1
```

Move `rear` :

```
rear = (1 + 1) % 5 = 2
```

Store value:

```
Q[2] = 20
```

Now:

```
front = 0  
rear = 2  
Logical queue = 10 20
```

31. Time Complexity of enqueue()

`enqueue()` does not use a loop.

It only:

- checks full condition
- moves `rear`
- stores one value

So the time complexity is:

```
O(1)
```

This means insertion is constant time.

32. The dequeue() Function

`dequeue()` means:

Delete and return the front value.

In a normal queue, deletion happens from the front.

Circular queue deletion also happens from the front, but `front` moves circularly.

33. Circular Queue dequeue() Code

```
int dequeue(struct Queue *q) {
    int x = -1;

    if (q->front == q->rear) {
        printf("Queue is Empty\n");
    } else {
        q->front = (q->front + 1) % q->size;
        x = q->Q[q->front];
    }

    return x;
}
```

34. Logic of dequeue()

To delete a value:

1. Create a variable `x` and set it to `-1`.
 2. Check whether the queue is empty.
 3. If the queue is empty, print `Queue is Empty`.
 4. If the queue is not empty, move `front` to the next circular position.
 5. Store `Q[front]` in `x`.
 6. Return `x`.
-

35. Why x Starts as -1

At the beginning of `dequeue()`, we write:

```
int x = -1;
```

This means:

```
If deletion fails, return -1.
```

For example, if the queue is empty, there is no value to delete.

So `-1` is returned as a signal.

36. Empty Check in dequeue()

The empty check is:

```
if (q->front == q->rear)
```

Simple meaning:

```
If front and rear are equal, there are no valid elements.
```

37. Moving front in dequeue()

This line moves `front`:

```
q->front = (q->front + 1) % q->size;
```

Simple meaning:

```
Move front one step forward.  
If front reaches the end, wrap it back to 0.
```

38. Getting the Deleted Value

After moving `front`, we read the value:

```
x = q->Q[q->front];
```

This value is the deleted value.

Example:

```
front = 1  
Q[1] = 10
```

Then:

```
x = 10
```

So **10** is removed and returned.

39. dequeue() Dry Run

Suppose the queue contains:

```
10 20
```

Current state:

```
size = 5  
front = 0  
rear = 2  
Q[1] = 10  
Q[2] = 20
```

Now call:

```
dequeue(&q);
```

Check empty:

```
front == rear
```

```
0 == 2
```

False, so the queue is not empty.

Move `front` :

```
front = (0 + 1) % 5 = 1
```

Get value:

```
x = Q[1] = 10
```

So `10` is returned.

Now:

```
front = 1  
rear = 2  
Logical queue = 20
```

40. Time Complexity of dequeue()

`dequeue()` does not shift elements.

It only:

- checks empty condition
- moves `front`
- returns one value

So the time complexity is:

```
O(1)
```

This means deletion is constant time.

41. Logical Deletion in Circular Queue

In a circular queue, deletion is also logical.

Logical deletion means:

The old value may still physically exist in the array, but it is no longer part of the queue.

Example:

Before deletion:

```
front = 0
rear = 2
Q[1] = 10
Q[2] = 20
Logical queue = 10 20
```

After deleting 10 :

```
front = 1
rear = 2
```

The array may still contain:

```
Q[1] = 10
Q[2] = 20
```

But the logical queue is only:

20

Why?

Because front has moved past 10 .

42. The display() Function

The display() function prints all valid elements in the queue.

Displaying a circular queue is more difficult than displaying a linear queue.

Why?

Because the elements may wrap around from the end of the array to the beginning.

43. Why Normal Display Loop Is Not Enough

In a linear queue, we could display like this:

```
for (int i = front + 1; i <= rear; i++) {  
    printf("%d ", Q[i]);  
}
```

But this does not always work for a circular queue.

Example:

Index:	0	1	2	3	4
Array:	50	60		30	40

The logical queue is:

30 40 50 60

Here, the queue starts near the end and continues at the beginning.

A normal loop from left to right may fail.

So we need circular display logic.

44. Circular Queue display() Code

```
void display(struct Queue q) {  
    if (q.front == q.rear) {  
        printf("Queue is Empty\n");  
        return;  
    }  
  
    int i = (q.front + 1) % q.size;  
  
    do {  
        printf("%d ", q.Q[i]);  
        i = (i + 1) % q.size;  
    }
```

```
    } while (i != (q.rear + 1) % q.size);  
  
    printf("\n");  
}
```

45. Logic of display()

To display a circular queue:

1. Check if the queue is empty.
2. If empty, print `Queue is Empty`.
3. Start from `(front + 1) % size`.
4. Print the value at the current index.
5. Move the index circularly using `(i + 1) % size`.
6. Stop after passing `rear`.

46. Why display() Starts at `(front + 1) % size`

In this design, `front` points before the first valid element.

So the first valid element is:

```
(front + 1) % size
```

That is why display starts with:

```
int i = (q.front + 1) % q.size;
```

47. Why display() Uses do-while

The display function uses a `do-while` loop because we want to print at least one element before checking the stopping condition.

```
do {  
    printf("%d ", q.Q[i]);  
    i = (i + 1) % q.size;  
} while (i != (q.rear + 1) % q.size);
```


The loop stops when `i` reaches the position after `rear` .

That position is:

```
(q.rear + 1) % q.size
```

48. Display Dry Run Without Wrap-Around

Suppose:

```
size = 5  
front = 0  
rear = 3  
Q[1] = 10  
Q[2] = 20  
Q[3] = 30
```

Start display at:

```
(front + 1) % size
```

```
(0 + 1) % 5 = 1
```

Print:

```
Q[1] = 10  
Q[2] = 20  
Q[3] = 30
```

Output:

```
10 20 30
```

49. Display Dry Run With Wrap-Around

Suppose:

```
size = 5  
front = 2  
rear = 1
```

The array is:

```
Index:  0   1   2   3   4  
Array: 50  60      30  40
```

The logical queue is:

```
30 40 50 60
```

Start display at:

```
(front + 1) % size
```

```
(2 + 1) % 5 = 3
```

Print index **3**:

```
30
```

Move circularly:

```
(3 + 1) % 5 = 4
```

Print index **4**:

```
40
```

Move circularly:

```
(4 + 1) % 5 = 0
```

Print index **0**:

50

Move circularly:

$(0 + 1) \% 5 = 1$

Print index :

60

So output is:

30 40 50 60

50. Complete Phase 6 Program in C

```
#include <stdio.h>
#include <stdlib.h>

struct Queue {
    int size;
    int front;
    int rear;
    int *Q;
};

void create(struct Queue *q, int size) {
    q->size = size;
    q->front = q->rear = 0;
    q->Q = (int *)malloc(q->size * sizeof(int));
}

void enqueue(struct Queue *q, int x) {
    if ((q->rear + 1) % q->size == q->front) {
        printf("Queue is Full\n");
    } else {
        q->rear = (q->rear + 1) % q->size;
        q->Q[q->rear] = x;
    }
}
```

```

int dequeue(struct Queue *q) {
    int x = -1;

    if (q->front == q->rear) {
        printf("Queue is Empty\n");
    } else {
        q->front = (q->front + 1) % q->size;
        x = q->Q[q->front];
    }

    return x;
}

void display(struct Queue q) {
    if (q.front == q.rear) {
        printf("Queue is Empty\n");
        return;
    }

    int i = (q.front + 1) % q.size;

    do {
        printf("%d ", q.Q[i]);
        i = (i + 1) % q.size;
    } while (i != (q.rear + 1) % q.size);

    printf("\n");
}

int main() {
    struct Queue q;

    create(&q, 5); // Can store only 4 elements because of the empty-slot rule

    enqueue(&q, 10);
    enqueue(&q, 20);
    enqueue(&q, 30);
    enqueue(&q, 40);

    display(q); // 10 20 30 40

    printf("Deleted: %d\n", dequeue(&q)); // 10
    printf("Deleted: %d\n", dequeue(&q)); // 20

    enqueue(&q, 50);
    enqueue(&q, 60); // rear wraps around if needed
}

```

```
    display(q); // 30 40 50 60

    free(q.Q);

    return 0;
}
```

51. Complete Program Explanation

Header files

```
#include <stdio.h>
#include <stdlib.h>
```

`stdio.h` is used for `printf()`.

`stdlib.h` is used for `malloc()` and `free()`.

Queue structure

```
struct Queue {
    int size;
    int front;
    int rear;
    int *Q;
};
```

This stores all queue information in one structure.

create function

```
create(&q, 5);
```

This creates a circular queue with array size `5`.

Because of the empty-slot rule, it can store only `4` values.

Insert values

```
enqueue(&q, 10);
enqueue(&q, 20);
```

```
enqueue(&q, 30);  
enqueue(&q, 40);
```

The queue becomes:

```
10 20 30 40
```

Delete values

```
dequeue(&q);  
dequeue(&q);
```

This removes:

```
10 and 20
```

The queue becomes:

```
30 40
```

Insert more values

```
enqueue(&q, 50);  
enqueue(&q, 60);
```

Now `rear` can wrap around to the beginning of the array.

The queue becomes:

```
30 40 50 60
```

Free memory

```
free(q.Q);
```

This releases the heap memory created by `malloc()`.

52. Expected Output

```
10 20 30 40
Deleted: 10
Deleted: 20
30 40 50 60
```

This output proves that the circular queue keeps FIFO order and also reuses spaces at the beginning of the array.

53. Full Program Dry Run

We start with:

```
create(&q, 5);
```

Because of the empty-slot rule:

```
size = 5
usable capacity = 4
```

Initial state:

```
front = 0
rear = 0
Queue is empty
```

Step 1: enqueue 10

```
enqueue(&q, 10);
```

Move `rear` :

```
rear = (0 + 1) % 5 = 1
```

Store:

```
Q[1] = 10
```

State:

```
front = 0  
rear = 1  
Logical queue = 10
```

Step 2: enqueue 20

```
enqueue(&q, 20);
```

Move `rear` :

```
rear = (1 + 1) % 5 = 2
```

Store:

```
Q[2] = 20
```

State:

```
front = 0  
rear = 2  
Logical queue = 10 20
```

Step 3: enqueue 30

```
enqueue(&q, 30);
```

Move `rear` :

```
rear = (2 + 1) % 5 = 3
```

Store:


```
Q[3] = 30
```

State:

```
front = 0  
rear = 3  
Logical queue = 10 20 30
```

Step 4: enqueue 40

```
enqueue(&q, 40);
```

Move `rear` :

```
rear = (3 + 1) % 5 = 4
```

Store:

```
Q[4] = 40
```

State:

```
front = 0  
rear = 4  
Logical queue = 10 20 30 40
```

Now the queue is full.

Why?

```
(rear + 1) % size == front
```

```
(4 + 1) % 5 == 0
```

```
0 == 0
```

So the queue is full.

Step 5: dequeue 10

```
dequeue(&q);
```

Move `front`:

```
front = (0 + 1) % 5 = 1
```

Deleted value:

```
Q[1] = 10
```

State:

```
front = 1  
rear = 4  
Logical queue = 20 30 40
```

Step 6: dequeue 20

```
dequeue(&q);
```

Move `front`:

```
front = (1 + 1) % 5 = 2
```

Deleted value:

```
Q[2] = 20
```

State:

```
front = 2
rear = 4
Logical queue = 30 40
```

Now indexes **1** and **2** have been passed by **front**.

They can be reused later.

Step 7: enqueue 50

```
enqueue(&q, 50);
```

Before insertion:

```
rear = 4
```

Move **rear**:

```
rear = (4 + 1) % 5 = 0
```

This is the wrap-around moment.

Store:

```
Q[0] = 50
```

State:

```
front = 2
rear = 0
Logical queue = 30 40 50
```

Step 8: enqueue 60

```
enqueue(&q, 60);
```

Move `rear` :

```
rear = (0 + 1) % 5 = 1
```

Store:

```
Q[1] = 60
```

State:

```
front = 2  
rear = 1  
Logical queue = 30 40 50 60
```

The array may look like this:

```
Index:  0   1   2   3   4  
Array: 50  60  20  30  40
```

But the logical queue is:

```
30 40 50 60
```

Why?

Because display starts after `front` and moves circularly.

54. Visual Pointer Table

Step	Operation	front	rear	Logical Queue
1	Create queue	0	0	Empty
2	Enqueue 10	0	1	10
3	Enqueue 20	0	2	10, 20
4	Enqueue 30	0	3	10, 20, 30
5	Enqueue 40	0	4	10, 20, 30, 40

Step	Operation	front	rear	Logical Queue
6	Dequeue	1	4	20, 30, 40
7	Dequeue	2	4	30, 40
8	Enqueue 50	2	0	30, 40, 50
9	Enqueue 60	2	1	30, 40, 50, 60

Important observation:

```
rear moved from 4 back to 0.
```

That is the main power of the circular queue.

55. Wrap-Around Moment

The most important moment happens here:

```
rear = (4 + 1) % 5;
```

This becomes:

```
rear = 0;
```

So instead of stopping at the last index, `rear` goes back to the beginning.

This solves the memory wastage problem.

56. How Circular Queue Solves Phase 5 Problem

In Phase 5, the linear queue had this problem:

```
Deleted spaces at the beginning could not be reused.
```

In Phase 6, circular queue solves it because:

rear can wrap around to the beginning.

Example:

After deleting 10 and 20, spaces near the beginning become reusable.

Then `rear` can move from index `4` to index `0`.

So the circular queue reuses array space better.

57. Circular Queue Does Not Break FIFO

Even though the array wraps around, the queue still follows FIFO.

Example insertion order:

10, 20, 30, 40, 50, 60

If we delete two values after inserting the first four, the first two deleted values are:

10, 20

That still follows FIFO.

The circular queue changes how memory is reused.

It does not change the queue rule.

58. Important Rule: Physical Array vs Logical Queue

The physical array may not look sorted in queue order.

Example:

Index:	0	1	2	3	4
Array:	50	60	20	30	40

But the logical queue is:

30 40 50 60

Why?

Because the queue order depends on `front`, `rear`, and circular movement.

The old value `20` may still physically exist at index `2`, but it is not part of the queue anymore.

This is logical deletion.

59. Complexity Summary

Operation	Time Complexity	Why?
<code>create()</code>	$O(1)$	Sets values and creates memory
<code>enqueue()</code>	$O(1)$	Checks full, moves <code>rear</code> , stores one value
<code>dequeue()</code>	$O(1)$	Checks empty, moves <code>front</code> , returns one value
<code>display()</code>	$O(n)$	Prints all valid elements

Space complexity:

$O(\text{size})$

The queue uses an array of the chosen size.

60. Why enqueue() and dequeue() Are $O(1)$

`enqueue()` is $O(1)$ because it only does a fixed number of actions:

Check full.
Move rear.
Store value.

`dequeue()` is $O(1)$ because it only does a fixed number of actions:

```
Check empty.  
Move front.  
Return value.
```

Neither function shifts elements.

61. Why display() Is $O(n)$

`display()` is $O(n)$ because it prints every valid queue element.

If the queue has 4 values, it prints 4 values.

If the queue has 100 values, it prints 100 values.

So display time depends on the number of elements.

62. Overflow in Circular Queue

Overflow means:

```
Trying to insert into a full queue.
```

In circular queue, the full condition is:

```
(rear + 1) % size == front
```

If this condition is true, insertion is not allowed.

The program prints:

```
Queue is Full
```

63. Underflow in Circular Queue

Underflow means:

Trying to delete from an empty queue.

In circular queue, the empty condition is:

```
front == rear
```

If this condition is true, deletion is not allowed.

The program prints:

Queue is Empty

The function returns:

-1

64. Key Terms From Phase 6

Term	Simple Meaning
Circular queue	Queue where array positions wrap around
Wrap-around	Moving from the last index back to index <code>0</code>
Modulo	Operator <code>%</code> used to calculate circular movement
Empty-slot rule	One array space is kept unused to distinguish full and empty
<code>front</code>	Index used for deletion
<code>rear</code>	Index used for insertion
<code>front == rear</code>	Empty condition
<code>(rear + 1) % size == front</code>	Full condition
Logical deletion	Deletion by moving <code>front</code> , even if old value remains in memory
Overflow	Trying to insert into a full queue
Underflow	Trying to delete from an empty queue

65. Common Beginner Mistakes

Mistake 1: Forgetting the Empty-Slot Rule

Wrong idea:

```
If size = 5, the queue can store 5 values.
```

Correct idea:

```
If size = 5, this circular queue can store only 4 values.
```

Why?

Because one slot is kept empty.

Mistake 2: Using `rear++` Instead of Modulo

Wrong:

```
rear++;
```

Problem:

```
rear may go outside the array.
```

Correct:

```
rear = (rear + 1) % size;
```

This keeps `rear` inside the valid index range.

Mistake 3: Using `front++` Instead of Modulo

Wrong:

```
front++;
```

Correct:

```
front = (front + 1) % size;
```

Both `front` and `rear` must move circularly.

Mistake 4: Confusing Full and Empty

Empty condition:

```
front == rear
```

Full condition:

```
(rear + 1) % size == front
```

Do not mix these two.

Mistake 5: Thinking `front` Points Directly to the First Element

In this design, `front` does not point directly to the first valid element.

It points before the first valid element.

The first valid element is:

```
(front + 1) % size
```

Mistake 6: Displaying With a Normal Linear Loop

Wrong idea:

```
for (int i = front + 1; i <= rear; i++)
```

Problem:

This fails when the queue wraps around.

Correct idea:

Use circular movement:

```
i = (i + 1) % size;
```

Mistake 7: Thinking the Physical Array Shows the Queue Order

The array may look like this:

```
50 60 20 30 40
```

But the logical queue may be:

```
30 40 50 60
```

Always follow `front`, `rear`, and circular movement.

Mistake 8: Forgetting to Free Memory

If memory is created using:

```
malloc()
```

Then release it using:

```
free(q.Q);
```

66. Phase 6 Mini Review Questions

Question 1

What problem does a circular queue solve?

Answer:

It solves the memory-wastage problem of a linear queue by allowing `rear` to wrap around and reuse spaces at the beginning.

Question 2

What formula is used to move circularly?

Answer:

```
(current + 1) % size
```

Question 3

What is the empty condition in this circular queue?

Answer:

```
front == rear
```

Question 4

What is the full condition in this circular queue?

Answer:

```
(rear + 1) % size == front
```

Question 5

If `size = 5`, how many values can this circular queue store?

Answer:

4 values

Because one slot is kept empty.

Question 6

If `rear = 4` and `size = 5`, what is the next rear position?

Answer:

$(4 + 1) \% 5 = 0$

So the next position is `0`.

Question 7

Does circular queue still follow FIFO?

Answer:

Yes. It still follows FIFO.

The first inserted value is still the first removed value.

Question 8

Why do we use modulo in a circular queue?

Answer:

Modulo helps the pointer return to index `0` after reaching the last index.

Question 9

Why is display harder in a circular queue?

Answer:

Because the queue elements may wrap around from the end of the array to the beginning.

Question 10

What is logical deletion?

Answer:

Logical deletion means the old value may still physically exist in the array, but it is no longer part of the queue because `front` moved past it.

67. Phase 6 Final Summary

In Phase 6, you learned the circular queue.

A circular queue solves the main problem of a linear queue:

memory wastage

In a linear queue, `rear` only moves forward.

So deleted spaces at the beginning cannot be reused.

In a circular queue, `rear` and `front` move using modulo:

$(\text{pointer} + 1) \% \text{size}$

This allows the queue to wrap around.

The circular queue starts with:

`front = rear = 0;`

The empty condition is:

```
front == rear
```

The full condition is:

```
(rear + 1) % size == front
```

This design uses the empty-slot rule, so the usable capacity is:

```
size - 1
```

You also learned:

- how to create a circular queue
- how to enqueue circularly
- how to dequeue circularly
- how to display circularly
- how wrap-around works
- how logical deletion still applies
- why circular queues still follow FIFO
- common beginner mistakes

The most important Phase 6 idea is:

Do not let the end of the array stop the queue.
Treat the array like a circle.

68. One-Line Memory Trick

Remember this:

```
Circular queue = normal queue + modulo movement.
```

And the key formula is:

```
(pointer + 1) % size
```

If you understand this formula, you understand the heart of Phase 6.